Report 4: IK2221 LLM inference project

Group 9: Riccardo Fragale, insert names

May 19, 2026

1 Task 1

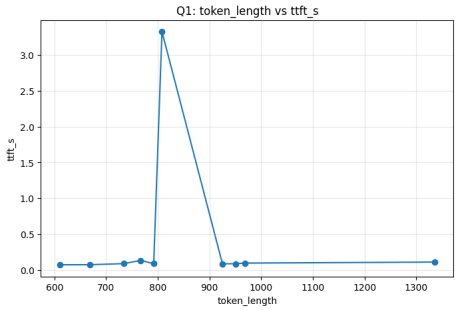
1.1 Question 1: Latency vs. Sequence Length

Evaluation methodology

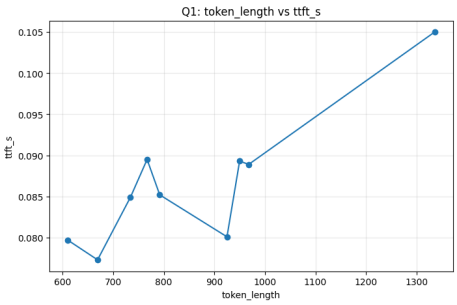
We decided to evaluate the impact of the sequence length on the prefill time (and in particular to the TTFT) with varying KV cache size (ranging from 0 to 4.0). We recorded TTFT for every request and the combined sequence length of the context of the question and the length of the prompt. The expectation is that TFFT grows sort of quadratically as the sequence length increases. Due to a cold start of the model we also saw that the first request arriving was particularly slow (ruining all metrics and averages), while the next ones have a similar pattern even though they involve a context switch. That's why we will show below some graphs regarding TTFT vs sequence length both including and not including the first request in order to better show what we found out. The benchmark provided (that could be added with –increasing-length) includes ten random questions about the different contexts, done in a way such that they have an increasing sequence length. This happens because the questions are selected inside three difference batches of prompts of different size. The first batch includes some short and general prompts while the other ones include medium size and large size prompts.

Results and analysis

In order not to avoid a list of 15 graphs we decide to include here the ones we believe are more meaningful but you can explore all the logs and all the graphs of our tests inside the folders `lmcache-vllm-extended\docker\results\q1` and also the `lmcache-vllm-extended\docker\results\q1_polished`. The polished ones include the graphs without the first request that, as said in the previous paragraph, ruins all averages and metrics and we haven't figured out why it is particularly slow. Let's first see what happens with cache size = 0 (so the model is not caching KV chunks actually).



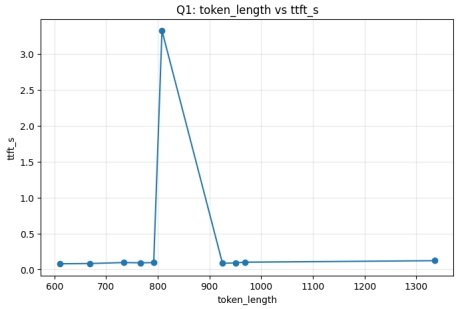
(a) Normal logs (q1)



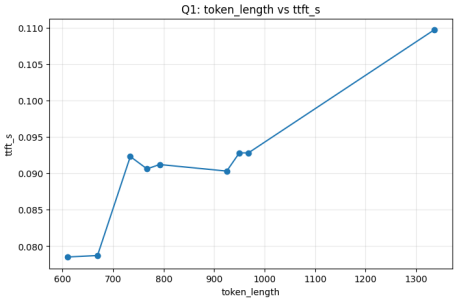
(b) Polished logs (q1_polished)

Figure 1: TTFT vs. sequence length for cache size = GB

The reason why we polished is the following:
Average TTFT: 0.4146 s.
Average TTFT excluding first request: 0.0913 s.
Analysing the polished graph it's clear that there is a trend that seems quite quadratic, which is what we expected.
Let's go now to cache size = 1.0 GB.



(a) Normal logs (q1)



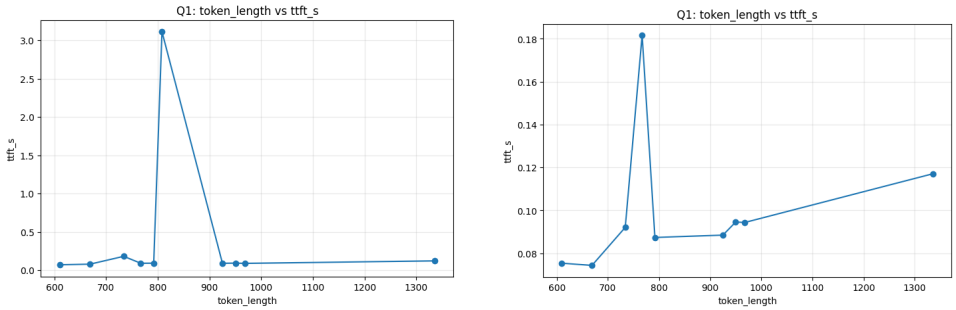
(b) Polished logs (q1_polished)

Figure 2: TTFT vs. sequence length for cache size = 1 GB

Average TTFT: 0.4156 s.
Average TTFT excluding first request: 0.0927 s.

There is again a linear to quadratic trend. Increasing the cache didn't generate a noticeable improvement in TTFT.

Finally let's try to increase the cache size to 4 GB.



(a) Normal logs (q1) (b) Polished logs (q1_polished)

Figure 3: TTFT vs. sequence length for cache size = 4 GB

Average TTFT: 0.4017 s.
Average TTFT excluding first request: 0.1004 s.

In this case we also notice that a request was particularly "slow" in the prefill despite it's not so long in terms of size of context and prompt. This indicates that maybe sometimes during a context switch some operations is randomly particularly slow.

To do a final summary, we have seen that the trend is quadratical (as expected) in almost each cache size as the impact of caching is not particularly relevant to reduce TTFT considering the sequence length. We expect to be meaningful regarding the next questions since it is more helpful with repeated requests about the same context as the KV remains in the GPu that is running the model.

1.2 Question 2: KV Cache Reuse and Cache Size Impact

Evaluation Methodology

To rigorously evaluate the latency improvements and cache sensitivity, we configured the LMCache-extended backend with varying levels of `max_local_cache_size` (ranging from 0.001 to 3.0). This parameter directly restricts or expands the available KV cache capacity. We then used a custom request generator running in a repeated sequence mode, querying the model with specific document contexts and repeating different questions within the same context before switching to a completely new context.

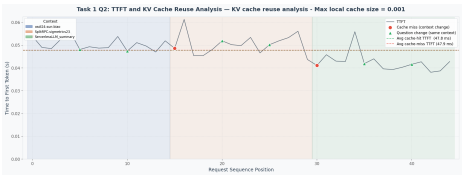
We recorded the Time To First Token (TTFT) for every request. By plotting the sequence of requests, we can visually identify cache misses (when a new context is loaded, requiring full prompt evaluation) and cache hits (when a new question is asked for a previously cached context). We calculated the average TTFT for hits and misses across each configuration to observe caching effectiveness.

Results and Analysis

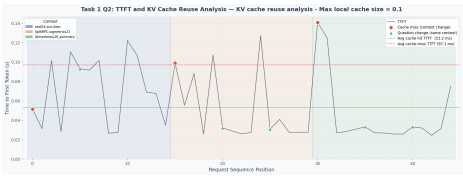
As shown in our timeline visualizations (Figure 4), feeding an old request to the model significantly reduces latency, provided the cache is large enough. The TTFT for cache hits (green markers) drops noticeably compared to cache misses (red markers), confirming that reusing the KV cache heavily optimizes inference time.

Furthermore, we analyzed how the size of the local cache affects these results. When the local cache size is extremely small (e.g., utilization = 0.001), the cache is unable to retain previously seen requests. This results in frequent evictions, causing identical latencies for both hits and misses, as the system is forced into recomputation.

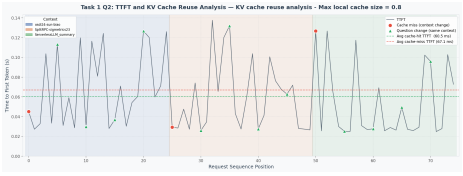
As the cache size increases (e.g., utilization ≥ 0.4), the system successfully stores the KV pairs. The gap between hit latency and miss latency widens significantly, showcasing the effectiveness of an adequately sized local cache. This overall trend is aggregated and clearly visible in Figure 5.



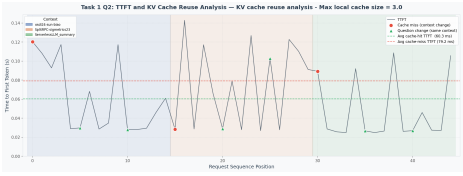
(a) Util = 0.001 (Frequent Evictions)



(b) Util = 0.1 (Partial Caching)



(c) Util = 0.8 (Stable Caching)



(d) Util = 3.0 (Oversized Cache)

Figure 4: Timeline visualizations of TTFT across varying GPU KV cache size. At extremely low utilization, the cache size is insufficient, causing hit and miss latencies to converge. At higher utilizations, cache hits show distinctly lower TTFT.

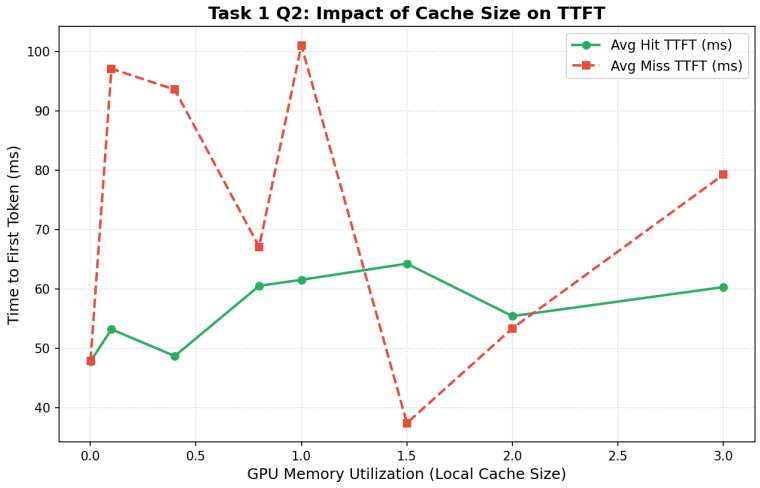


Figure 5: Impact of KV cache size on Average TTFT. The gap between misses and hits demonstrates caching efficiency.

1.3 Question 3: Impact of Request Diversity

2 Task 2

3 Task 3

4 Task 4